

Resolução das Simulações: Filtros Digitais

Este documento contém a implementação em Python das 10 questões de simulação propostas no estudo dirigido, utilizando as bibliotecas numpy, scipy.signal e matplotlib. Os espaços marcados devem ser preenchidos com as respectivas capturas de tela dos gráficos gerados em sua máquina.

1. Filtragem de Senoides (Filtro Passa-Baixa)

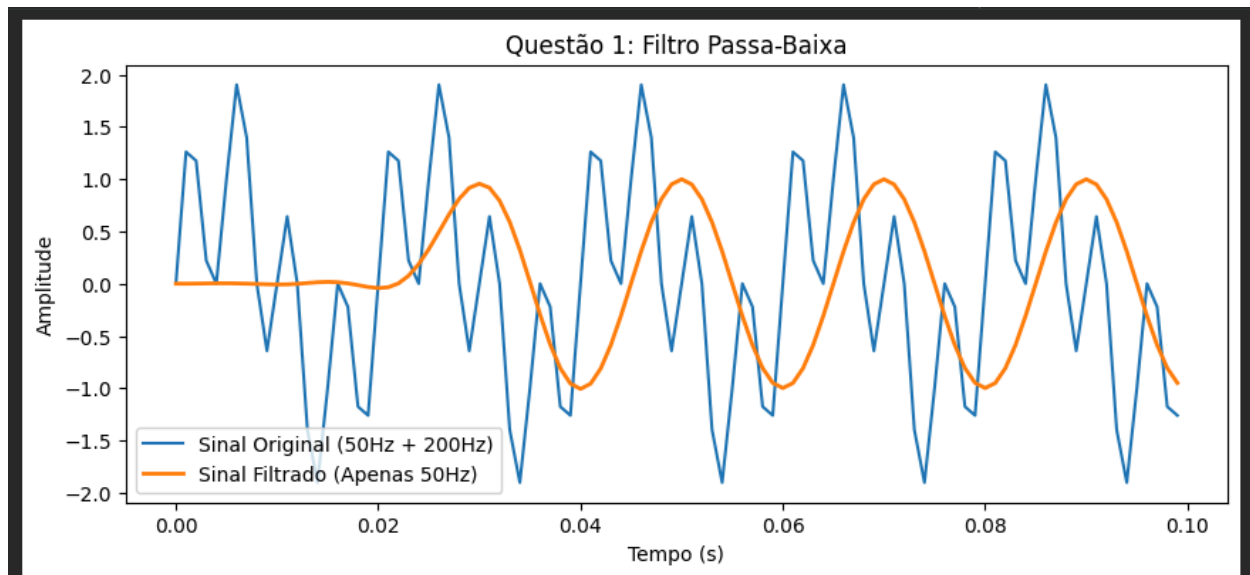
Objetivo: Gerar um sinal composto por duas senoides (ex: 50 Hz e 200 Hz) e aplicar um filtro passa-baixa (corte em 100 Hz) para preservar apenas a componente de baixa frequência.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

fs = 1000 # Frequência de amostragem
t = np.arange(0, 1, 1/fs)
sinal = np.sin(2 * np.pi * 50 * t) + np.sin(2 * np.pi * 200 * t)

# Projeto do filtro FIR passa-baixa (corte = 100 Hz)
nyq = 0.5 * fs
taps = signal.firwin(51, 100/nyq)
sinal_filtrado = signal.lfilter(taps, 1.0, sinal)

plt.figure(figsize=(10, 4))
plt.plot(t[:100], sinal[:100], label='Sinal Original (50Hz + 200Hz)')
plt.plot(t[:100], sinal_filtrado[:100], label='Sinal Filtrado (Apenas 50Hz)', linewidth=2)
plt.legend()
plt.title('Questão 1: Filtro Passa-Baixa')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude')
plt.show()
```



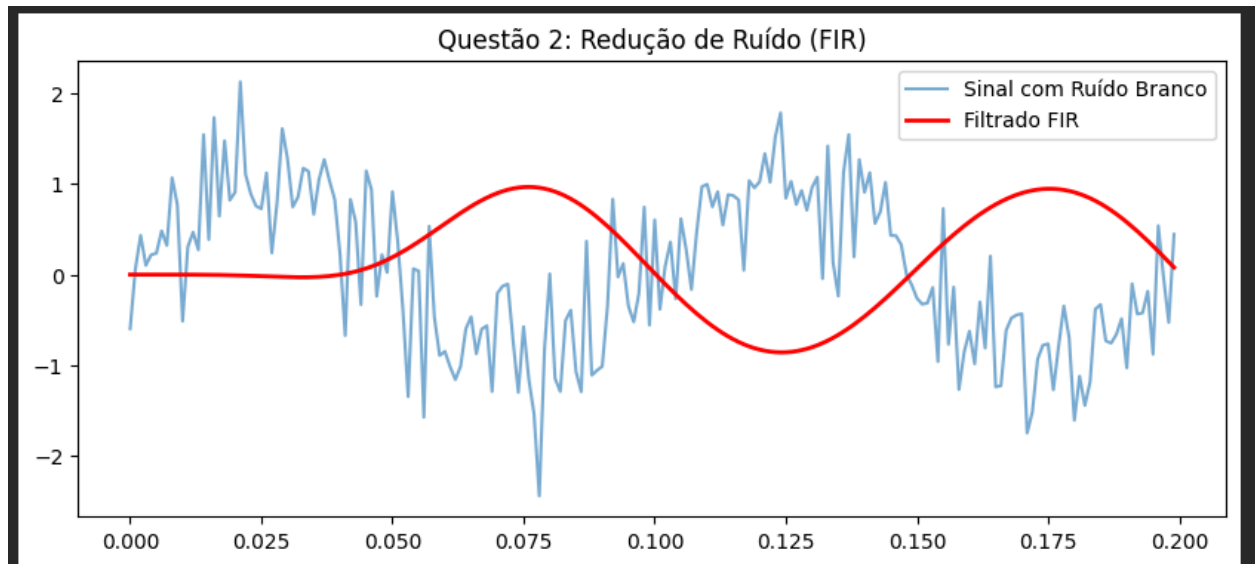
2. Redução de Ruído com Filtro FIR

Objetivo: Adicionar ruído branco a um sinal limpo e utilizar um filtro FIR para suavizá-lo.

```
sinal_limpo = np.sin(2 * np.pi * 10 * t)
ruído = np.random.normal(0, 0.5, len(t))
sinal_ruidoso = sinal_limpo + ruído

taps_fir = signal.firwin(101, 20/nyq) # Corte em 20Hz
sinal_filtrado_fir = signal.lfilter(taps_fir, 1.0, sinal_ruidoso)

plt.figure(figsize=(10, 4))
plt.plot(t[:200], sinal_ruidoso[:200], label='Sinal com Ruído Branco', alpha=0.6)
plt.plot(t[:200], sinal_filtrado_fir[:200], label='Filtrado FIR', color='red', linewidth=2)
plt.legend()
plt.title('Questão 2: Redução de Ruído (FIR)')
plt.show()
```

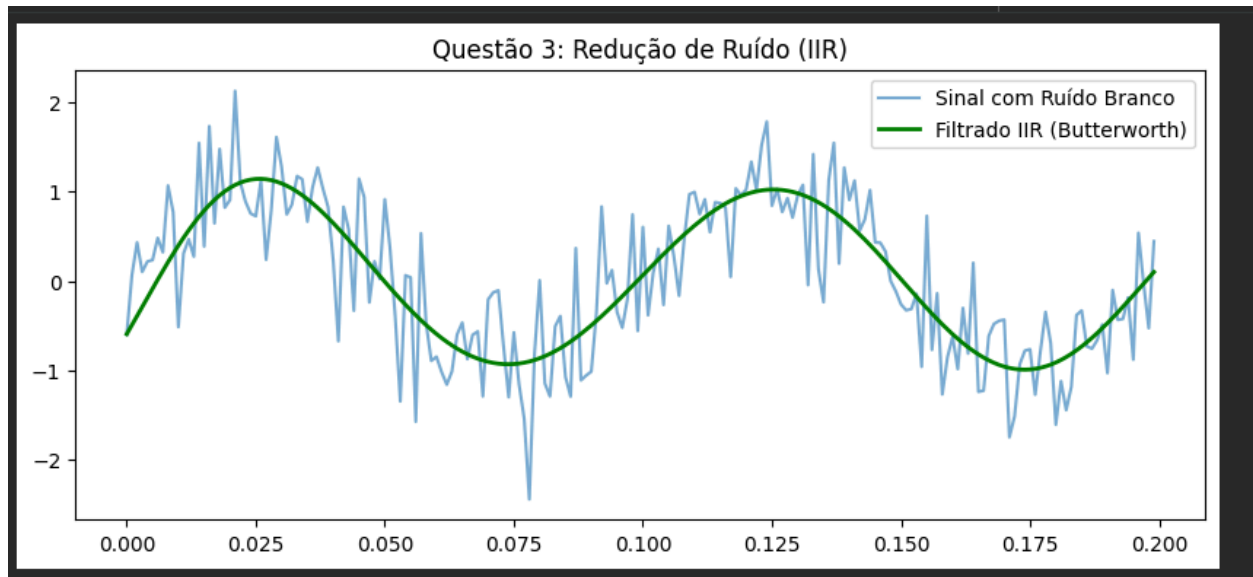


3. Redução de Ruído com Filtro IIR (Butterworth)

Objetivo: Repetir o experimento da questão anterior utilizando um filtro IIR Butterworth.

```
# Projeto do filtro IIR Butterworth (Ordem 4, corte 20Hz)
b, a = signal.butter(4, 20/nyq, btype='low')
sinal_filtrado_iir = signal.filtfilt(b, a, sinal_ruidoso) #
filtfilt para fase zero

plt.figure(figsize=(10, 4))
plt.plot(t[:200], sinal_ruidoso[:200], label='Sinal com Ruído
Branco', alpha=0.6)
plt.plot(t[:200], sinal_filtrado_iir[:200], label='Filtrado IIR
(Butterworth)', color='green', linewidth=2)
plt.legend()
plt.title('Questão 3: Redução de Ruído (IIR)')
plt.show()
```

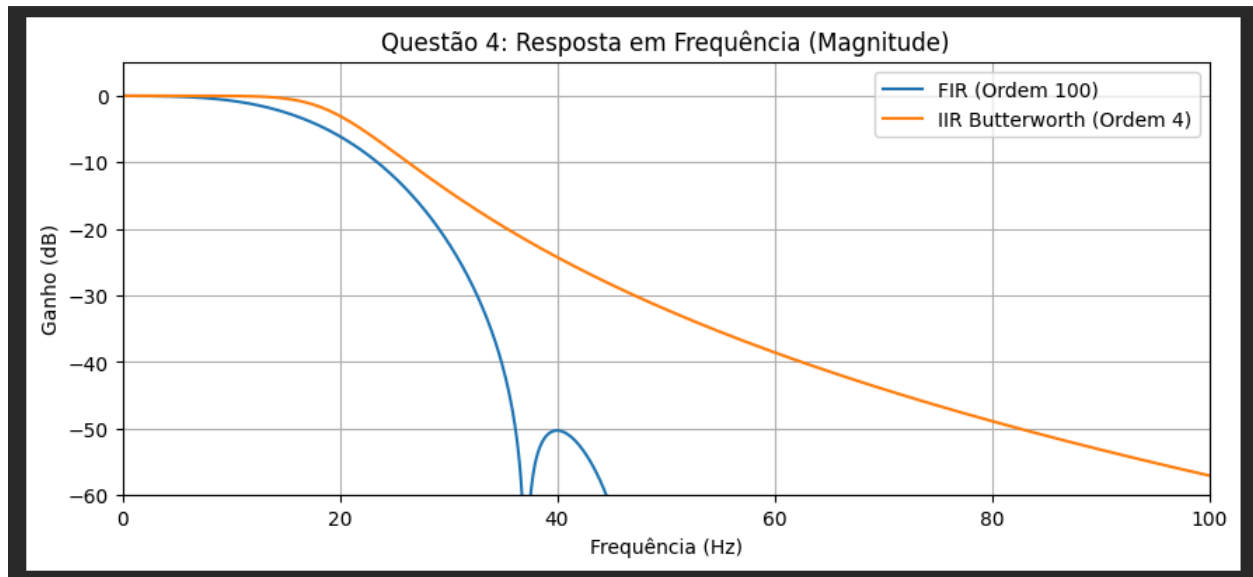


4. Comparação de Resposta em Frequência (FIR vs IIR)

Objetivo: Comparar a resposta em magnitude de filtros FIR e IIR com frequências de corte semelhantes.

```
w_fir, h_fir = signal.freqz(taps_fir, 1.0, worN=8000)
w_iir, h_iir = signal.freqz(b, a, worN=8000)

plt.figure(figsize=(10, 4))
plt.plot(0.5*fs*w_fir/np.pi, 20 * np.log10(np.abs(h_fir)),
label='FIR (Ordem 100)')
plt.plot(0.5*fs*w_iir/np.pi, 20 * np.log10(np.abs(h_iir)),
label='IIR Butterworth (Ordem 4)')
plt.xlim(0, 100)
plt.ylim(-60, 5)
plt.legend()
plt.title('Questão 4: Resposta em Frequência (Magnitude)')
plt.xlabel('Frequência (Hz)')
plt.ylabel('Ganho (dB)')
plt.grid(True)
plt.show()
```

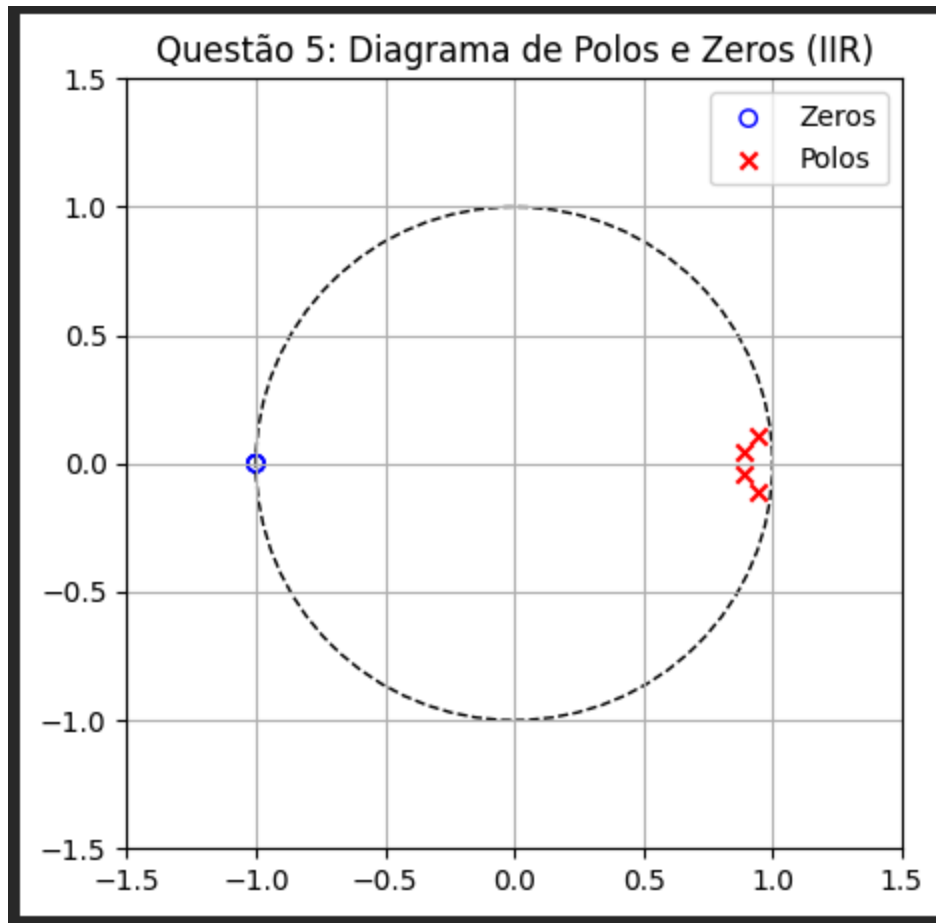


5. Polos e Zeros (Estabilidade)

Objetivo: Plotar o diagrama de polos e zeros do filtro IIR projetado para avaliar sua estabilidade.

```
z, p, k = signal.tf2zpk(b, a)

plt.figure(figsize=(5, 5))
plt.scatter(np.real(z), np.imag(z), marker='o', edgecolors='b',
            facecolors='none', label='Zeros')
plt.scatter(np.real(p), np.imag(p), marker='x', color='r',
            label='Polos')
circulo = plt.Circle((0, 0), 1, color='black', fill=False,
                    linestyle='--')
plt.gca().add_patch(circulo)
plt.xlim([-1.5, 1.5])
plt.ylim([-1.5, 1.5])
plt.grid(True)
plt.legend()
plt.title('Questão 5: Diagrama de Polos e Zeros (IIR)')
plt.show()
```

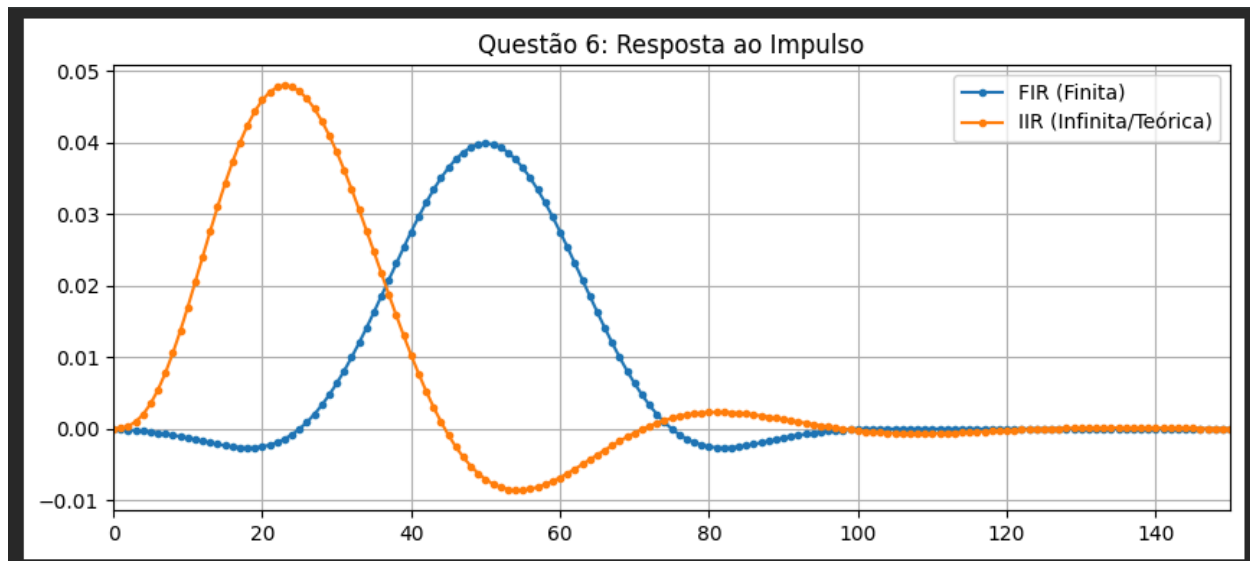


6. Resposta ao Impulso (FIR vs IIR)

Objetivo: Analisar a duração da resposta ao impulso entre os dois tipos de arquitetura.

```
impulso = signal.unit_impulse(200)
resp_imp_fir = signal.lfilter(taps_fir, 1.0, impulso)
resp_imp_iir = signal.lfilter(b, a, impulso)

plt.figure(figsize=(10, 4))
plt.plot(resp_imp_fir, label='FIR (Finita)', marker='.')
plt.plot(resp_imp_iir, label='IIR (Infinita/Teórica)', marker='.')
plt.legend()
plt.title('Questão 6: Resposta ao Impulso')
plt.xlim(0, 150)
plt.grid(True)
plt.show()
```

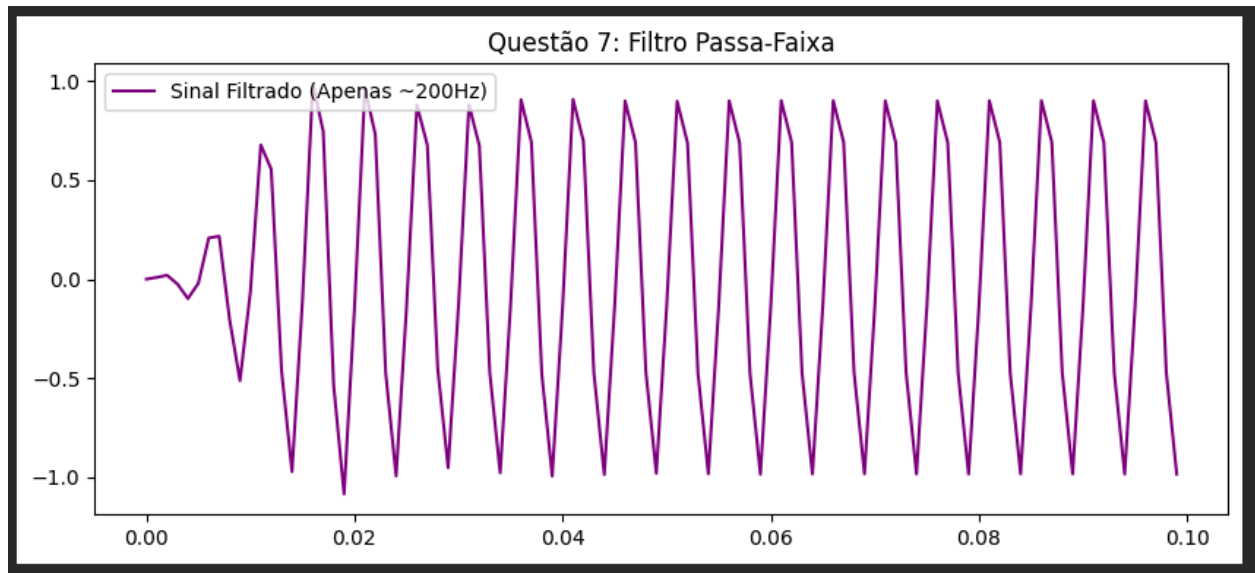


7. Filtro Passa-Faixa

Objetivo: Isolar uma frequência específica (ex: 200Hz) de um sinal complexo.

```
sinal_complexo = np.sin(2 * np.pi * 50 * t) + np.sin(2 * np.pi * 200 * t) + np.sin(2 * np.pi * 400 * t)
b_pf, a_pf = signal.butter(4, [150/nyq, 250/nyq], btype='bandpass')
sinal_pf = signal.lfilter(b_pf, a_pf, sinal_complexo)

plt.figure(figsize=(10, 4))
plt.plot(t[:100], sinal_pf[:100], label='Sinal Filtrado (Apenas ~200Hz)', color='purple')
plt.legend()
plt.title('Questão 7: Filtro Passa-Faixa')
plt.show()
```

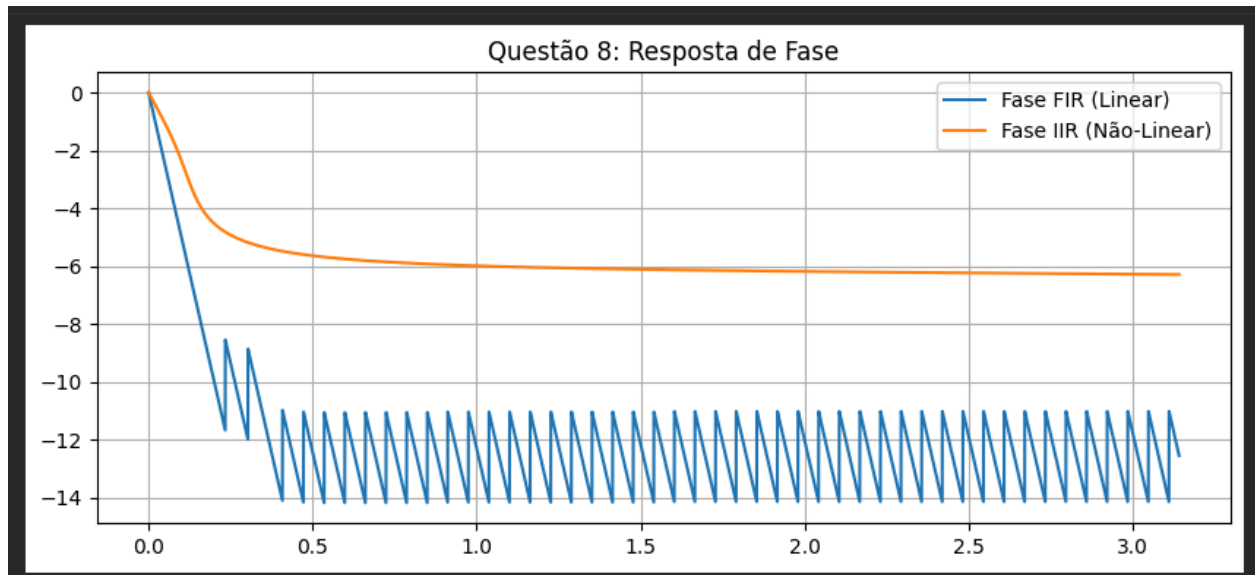


8. Resposta de Fase

Objetivo: Evidenciar a linearidade de fase do FIR versus a não-linearidade do IIR.

```
fase_fir = np.unwrap(np.angle(h_fir))
fase_iir = np.unwrap(np.angle(h_iir))

plt.figure(figsize=(10, 4))
plt.plot(w_fir, fase_fir, label='Fase FIR (Linear)')
plt.plot(w_iir, fase_iir, label='Fase IIR (Não-Linear)')
plt.legend()
plt.title('Questão 8: Resposta de Fase')
plt.grid(True)
plt.show()
```

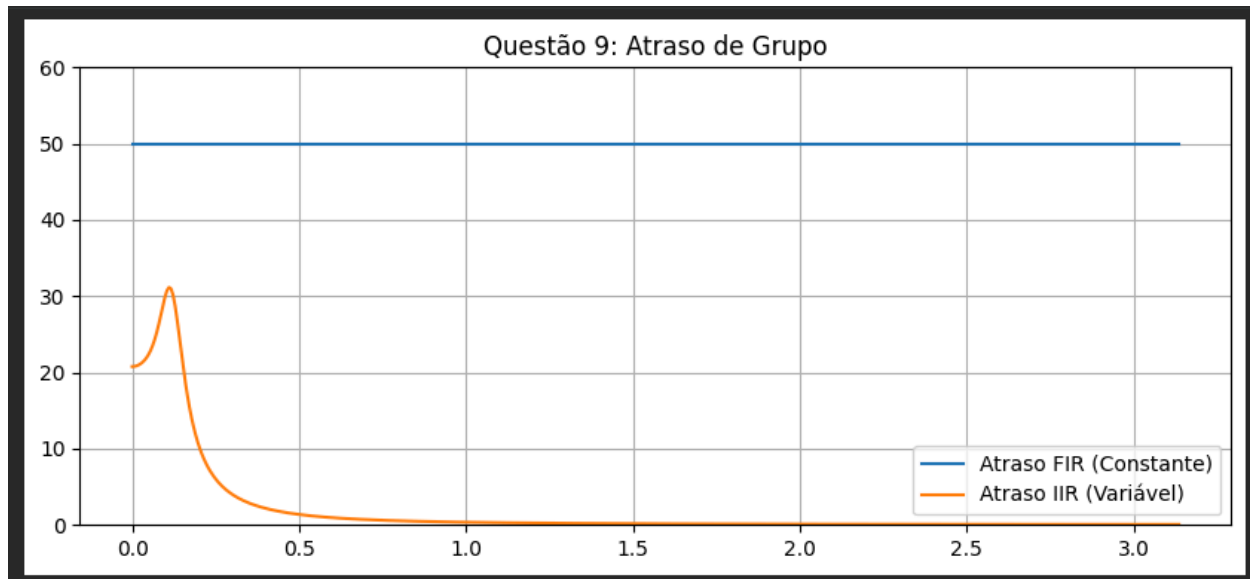



9. Atraso de Grupo

Objetivo: Calcular o atraso de grupo, demonstrando que o FIR possui atraso constante na banda de passagem.

```
w, gd_fir = signal.group_delay((taps_fir, 1.0))
w, gd_iir = signal.group_delay((b, a))

plt.figure(figsize=(10, 4))
plt.plot(w, gd_fir, label='Atraso FIR (Constante)')
plt.plot(w, gd_iir, label='Atraso IIR (Variável)')
plt.legend()
plt.title('Questão 9: Atraso de Grupo')
plt.ylim(0, 60)
plt.grid(True)
plt.show()
```



10. Aplicação Prática: Suavização de Sensores (TinyML / Hardware)

Objetivo: Simular o condicionamento de um sinal ruidoso de um sensor (como um termopar ou acelerômetro em um sistema agrícola) usando uma média móvel (um tipo de filtro FIR). Este processo é vital antes de alimentar os dados para uma rede neural ou algoritmo de controle em microcontroladores (como a arquitetura Cortex-M3).

```
# Simulação de um sensor agrícola lendo temperatura com ruído de
campo
temp_real = 25.0 + 5.0 * np.sin(2 * np.pi * 0.1 * t)
ruído_sensor = np.random.normal(0, 1.5, len(t))
leitura_sensor = temp_real + ruído_sensor

# Filtro de Média Móvel Simples (N=20 amostras)
janela = 20
filtro_media = np.ones(janela) / janela
leitura_suavizada = signal.lfilter(filtro_media, 1.0,
leitura_sensor)

plt.figure(figsize=(10, 4))
plt.plot(t, leitura_sensor, color='lightgray', label='Leitura Bruta
do Sensor')
plt.plot(t, temp_real, color='black', linestyle='--',
```

```
label='Temperatura Real')
plt.plot(t, leitura_suavizada, color='orange', linewidth=2.5,
label='Sinal Condicionado (Média Móvel)')
plt.legend()
plt.title('Questão 10: Condicionamento de Sinal de Sensor')
plt.xlabel('Tempo')
plt.ylabel('Temperatura (°C)')
plt.show()
```

